

Rapport cryptographie

Version PDF générée à partir du document Word original.

Sous la direction :

Mr Fall

Réalisé Par :

Mouhamadou Falilou Sarr

Ndeye Mariam Niang

RAPPORT

Cryptographie

Mai 2024

Exercice 1 :

1. Quelles sont les exigences fondamentales en sécurité informatique ?

Les exigences fondamentales en sécurité informatique comprennent la confidentialité, l'intégrité, la disponibilité, l'authenticité, la non-répudiation et la traçabilité des informations.

2. Citez quatre (4) services de sécurité et quatre (4) mécanismes de sécurité définis dans X.800 :

Les services de sécurité incluent l'authentification, l'autorisation, la confidentialité et l'intégrité. Les mécanismes de sécurité comprennent le chiffrement symétrique, le chiffrement asymétrique, les fonctions de hachage et les protocoles d'échange de clés.

3. Quelles sont les principales différences entre RSA et AES en termes de fonctionnalité et d'utilisation ?

RSA est un algorithme de cryptographie asymétrique utilisé pour l'authentification, la signature numérique et l'échange de clés. Il utilise une paire de clés (publique et privée) pour chiffrer et déchiffrer les données. Les clés RSA sont généralement plus grandes et le processus est plus lent que celui d'AES. AES, en revanche, est un algorithme de cryptographie symétrique utilisé principalement pour le chiffrement et le déchiffrement de données sensibles. Il utilise la même clé pour les deux opérations et est plus rapide et efficace que RSA. Les clés AES sont plus courtes que celles de RSA.

4. Expliquez le fonctionnement de l'algorithme RSA en détail, en incluant les étapes clés telles que la génération de clés, le chiffrement et le déchiffrement :

L'algorithme RSA (Rivest-Shamir-Adleman) est un algorithme de cryptographie asymétrique largement utilisé pour l'authentification, la signature numérique et le chiffrement. Voici comment il fonctionne en détail :

Génération des clés :

- Sélection de deux nombres premiers distincts, généralement désignés par p et q .
- Calcul du produit de ces deux nombres, $n = p * q$, qui constitue la partie publique de la clé.
- Calcul de la fonction d'Euler de n , notée $\phi(n)$.
- Sélection d'un entier e , qui est relativement premier à $\phi(n)$. Cet entier constitue la partie publique de la clé.

- Calcul de l'entier d , qui est l'inverse multiplicatif de e modulo $\phi(n)$. L'entier d constitue la partie privée de la clé.
- La clé publique est constituée de n et e , tandis que la clé privée est constituée de n et d .

Chiffrement :

- Pour chiffrer un message M en utilisant la clé publique (n, e) , le message est converti en un entier m tel que $0 \leq m < n$.
- Le message chiffré C est obtenu en calculant $C = m^e \bmod n$.
- Le message chiffré C peut être envoyé en toute sécurité au destinataire.

Déchiffrement :

- Le destinataire reçoit le message chiffré C et utilise sa clé privée (n, d) pour le déchiffrer.
- Pour déchiffrer C , le destinataire calcule $m = C^d \bmod n$.
- Le destinataire obtient ainsi l'entier m , qui est converti en texte clair pour récupérer le message M d'origine.

5. Expliquez le mode de fonctionnement AES-CBC avec un schéma illustratif :

Le mode de chiffrement AES-CBC (Cipher Block Chaining) utilise un schéma où chaque bloc de texte clair est combiné avec le bloc chiffré précédent avant le chiffrement. Cela crée un effet de chaînage qui augmente la sécurité du chiffrement. Chaque bloc chiffré dépend du bloc précédent, rendant le chiffrement plus sûr.

6. Comment la sécurité des courbes elliptiques est évaluée et comment elle se compare à celle de RSA ?

La sécurité des courbes elliptiques est évaluée en fonction de la taille des paramètres utilisés dans l'algorithme. Plus la taille est grande, plus il est difficile pour une attaque de force brute de réussir. Les courbes elliptiques offrent une sécurité comparable à celle de RSA, mais avec des clés beaucoup plus courtes, ce qui les rend souvent plus efficaces en termes de performances et de stockage. La comparaison exacte dépend des paramètres spécifiques utilisés dans chaque algorithme et des avancées dans les méthodes d'attaque.

7. Quelles sont les propriétés des fonctions de hachage :

Déterminisme : Pour une entrée donnée, la fonction de hachage doit toujours produire la même sortie.

Rapide à calculer : La fonction de hachage doit être rapide à calculer pour ne pas ralentir le processus.

Résistance à la pré-image : Il doit être difficile de trouver l'entrée à partir d'une sortie donnée.

Résistance à la seconde pré-image : Il doit être difficile de trouver une deuxième entrée qui produit la même sortie.

Résistance aux collisions : Il doit être difficile de trouver deux entrées différentes qui produisent la même sortie.

8. Décrivez les mécanismes de sécurité d'un réseau privé virtuel (VPN) utilisant le protocole IPSec.

Un VPN utilisant le protocole IPSec utilise deux principaux mécanismes de sécurité : l'Authentication Header (AH) et le Encapsulating Security Payload (ESP). AH assure l'intégrité et l'authenticité des paquets IP en ajoutant un en-tête qui contient un code d'authentification du message (MAC). ESP, quant à lui, fournit la confidentialité en chiffrant les données du paquet IP.

9. Comparez les modes de fonctionnement Transport et Tunnel d'IPSec.

Le mode Transport d'IPSec sécurise les données entre deux hôtes, tandis que le mode Tunnel sécurise les données entre deux passerelles VPN. Le mode Transport protège seulement le contenu du paquet IP, tandis que le mode Tunnel protège tout le paquet IP, y compris les en-têtes.

10. Quel est le rôle de IKE (Internet Key Exchange) dans IPSec ?

IKE est utilisé pour établir les paramètres de sécurité IPSec, notamment les clés de chiffrement et les méthodes d'authentification, entre les deux extrémités d'une connexion VPN.

11. Décrivez les principaux composants de IPSec : AH et ESP.

AH (Authentication Header) fournit l'intégrité et l'authenticité des paquets IP en utilisant des codes d'authentification de message (MAC). ESP (Encapsulating Security Payload) fournit la confidentialité des données en chiffrant les paquets IP.

12. Qu'est-ce qu'une SA et à quoi sert-il dans le contexte de IPSec ?

Une SA (Security Association) est un ensemble de paramètres de sécurité, tels que les algorithmes de chiffrement et d'authentification, associés à une communication unidirectionnelle entre deux entités IPSec. Il est utilisé pour sécuriser le trafic dans le contexte d'IPSec.

13. Quelles sont les différences entre SAD et SPD dans IPSec ?

SAD (Security Association Database) est une base de données qui stocke les informations spécifiques à chaque SA, telles que les clés et les paramètres de chiffrement. SPD (Security Policy Database) est une base de données qui définit les politiques de sécurité pour le traitement du trafic IP, telles que les règles de filtrage et les actions à prendre sur le trafic.

14. Faites un schéma explicatif du processus d'établissement d'une connexion sécurisée SSL/TLS entre un client et un serveur. Incluez les étapes de négociation de la sécurité, de vérification du certificat et d'échange de clés :

Serveur

Client

ServeurHello

Client Hello

ServeurKeyExchange

ServeurHelloDone

Certificat (si requis)

Et

ClientKeyExchange

ChangeCipherSpec

ChangeCipherSpec

Finished

Finished

Le client envoie un message ClientHello contenant les paramètres de la connexion et les algorithmes de chiffrement qu'il supporte.

Le serveur répond avec un message ServerHello, qui contient les paramètres de la connexion choisis parmi ceux proposés par le client, ainsi qu'un certificat numérique contenant la clé publique du serveur.

Le serveur envoie un message ServerKeyExchange, qui contient des informations supplémentaires nécessaires à l'échange de clés si nécessaire.

Le serveur envoie un message ServerHelloDone pour indiquer qu'il a terminé la phase d'initialisation.

Le client vérifie le certificat du serveur, génère une clé de session, et envoie son propre certificat s'il est requis, ainsi que le message ClientKeyExchange.

Les deux parties échangent un message ChangeCipherSpec pour activer le chiffrement, puis envoient un message Finished pour confirmer que la connexion est établie de manière sécurisée.

15. Discutez des implications de l'évolution des technologies de cryptographie sur la sécurité des réseaux LAN, en tenant compte des avancées récentes telles que les algorithmes de chiffrement quantique :

L'évolution des technologies de cryptographie, notamment l'avènement des algorithmes de chiffrement quantique, présente des implications majeures pour la sécurité des réseaux LAN. Bien que les algorithmes quantiques offrent un potentiel de cryptographie inviolable face aux attaques des ordinateurs quantiques, leur adoption généralisée est entravée par des défis tels que la complexité de mise en œuvre et les coûts élevés. Pour garantir la sécurité à long terme, il est essentiel de maintenir l'utilisation de protocoles et d'algorithmes de cryptographie éprouvés, tout en explorant de nouvelles méthodes de cryptographie post-quantique.

Tp1 : LE HACHAGE

1. Expliquez la différence entre le hachage, le chiffrement et la signature :

Le hachage, le chiffrement et la signature sont des techniques cryptographiques distinctes. Le hachage transforme une donnée en une empreinte unique, utile pour vérifier l'intégrité des données. Le chiffrement convertit les données en une forme illisible pour protéger leur confidentialité, tandis que la signature numérique utilise une clé privée pour créer une signature attachée à un message, vérifiable par quiconque possède la clé publique correspondante, assurant ainsi l'authenticité et l'intégrité du message. En bref, le hachage assure l'intégrité, le chiffrement la confidentialité, et la signature l'authentification et la non-répudiation.

2. Calculez le Hash (MD5, SHA256, SHA512) du message avec :

Openssl :

MD5 :

SHA256 :

SHA512 :

MD5 :

Certutil :

MD5 :

SHA256 :

SHA512 :

Get-FileHash :

MD5 :

SHA256 :

SHA512 :

3. Combien de temps faudrait-il, en moyenne, pour qu'une collision se produise dans une fonction de hachage, connaissant la taille de l'espace des hachages possibles exprimée en bits ?

Le temps moyen pour qu'une collision se produise dans une fonction de hachage dépend de la taille de l'espace des hachages possibles exprimée en bits et du nombre de hash générés. En utilisant l'anniversaire paradoxal, on estime que le nombre d'opérations nécessaires pour qu'une collision se produise est d'environ la racine carrée de la taille de l'espace des hachages possibles. Pour calculer le temps moyen, on peut utiliser la formule suivante :

Temps moyen = $\frac{2^{\text{nombre de bits}}}{2 \times \text{nombre de hash générés par seconde} \times 365 \times 24 \times 60 \times 60}$

2 x nombre de hash générés par seconde x 365 x 24 x 60 x 60

4. Téléchargez et Vérifiez l'intégrité d'une image iso Ubuntu 20 :

Téléchargeons l'image Ubuntu 20 :

On clique sur fichier de hachage l'image Ubuntu 20 et on copie la valeur de hachage :

Vérifions l'intégrité de l'image ISO en exécutant la commande `CertUtil -hashfile "ubuntu-20.04.6-desktop-amd64.iso" SHA256` :

Comparons la valeur de hachage SHA256 affichée par la commande avec la valeur de hachage correspondante dans le fichier SHA256SUMS :

Les deux valeurs sont identiques !

Tp2 : Chiffrement et signature avec l'algorithme RSA

Partie 1 : Chiffrement et déchiffrement

1. Générons une paire de clés RSA de 4096 bits avec openssl au format PEM :

Générons la clé privée avec la commande : `openssl genrsa -out private_key.pem 4096`

Générons la clé publique avec la commande : `openssl rsa -pubout -in private_key.pem -out public_key.pem`

2. Créons le fichier message.txt avec un message en clair :

3. Chiffrons le message avec la clé publique avec la commande : `openssl rsautl -encrypt -pubin -inkey public_key.pem -in message.txt -out message_chiffre.txt`

4. Affichons le message chiffré :

5. Déchiffrons le message avec la clé privée avec la commande : `openssl rsautl -decrypt -inkey private_key.pem -in message_chiffre.txt`

Partie 2 : Signature

1. Faisons un schéma explicatif de la signature numérique :

Message

Message

Hachage du message

Hachage du message

Signature du hash avec la clé publique

Signature du hash avec la clé privée

Message vérifié

Message signé

2. Signons un message :

Créons le fichier avec notre message :

Générons le hash du message et copions le dans un fichier avec la commande : `openssl dgst -sha256 -binary < message.txt > message.sha256`

Utilisons la clé privée signer le message avec la commande : `openssl rsautl -sign -inkey private_key.pem -in message.sha256 -out signature.sha256`

3. Vérifions le message :

Utilisons la clé publique pour déchiffrer le hash générer et vérifier la signature avec la commande : `openssl rsautl -verify -inkey public_key.pem -pubin -in signature.sha256 -out signature_dechiffre.sha256`

Comparons les deux hash à l'aide de la commande : `diff -s message.sha256 signature_dechiffre.sha256`

4. Modifions le message et revérifions la signature :

Modifions le message dans le fichier message.txt :

Générons le nouveau hash du message avec la commande : `openssl dgst -sha256 -binary < message.txt > new_message_sha256`

Signons à nouveau le message avec la clé privée à l'aide de la commande : `openssl rsautl -sign -inkey private_key.pem -in new_message_sha256 -out new_signature.sha256`

Comparons les deux hash à l'aide de la commande : `diff -s signature.sha256 new_signature.sha256`

5. Que constatez-vous ?

On constate que les deux hash sont différents par ce que le message a été modifié. Et donc toute altération du message invalide la signature associée à ce message.

Partie 3 : Les courbes elliptiques

1. Listez les courbes elliptiques supportées par openssl avec la commande `openssl ecparam -list_curves` :

2. Créons une paire de clés cryptographiques basée sur les courbes elliptiques avec openssl :

Générons une clé de session avec la commande :

```
openssl ecparam -genkey -name prime256v1 -out ec_private_key.pem
```

Générons la clé publique avec la commande : `openssl ec -in ec_private_key.pem -pubout -out ec_public_key.pem`

Générons une clé symétrique aléatoire avec la commande : `openssl rand -out session_key.bin 32`

3. Chiffrons/déchiffrons un message et signons/vérifions une signature avec la paire de clés et la clé symétrique aléatoires :

Chiffrons le message avec la commande : `openssl enc -aes-256-cbc -salt -in message.txt -out encrypted_message.bin -pass file:session_key.bin`

Déchiffrons le message avec la commande : `openssl enc -d -aes-256-cbc -in encrypted_message.bin -out decrypted_message.txt -pass file:session_key.bin`

Signons le message avec la commande : `openssl dgst -sha256 -sign ec_private_key.pem -out message.sha256 message.txt`

Vérifions la signature du message avec la commande : `openssl dgst -sha256 -verify ec_public_key.pem -signature message.sha256 message.txt`

Tp3 : Chiffrement symétrique

1. Listons les algorithmes de chiffrement symétrique supportés par openssl avec la commande : `openssl list -cipher-algorithms`

2. Différence entre la clé de chiffrement et l'algorithme chiffrement dans la cryptographie symétrique :

La clé de chiffrement est une donnée secrète utilisée pour chiffrer et déchiffrer des données dans un algorithme de chiffrement symétrique, tandis que l'algorithme de chiffrement est la méthode ou le protocole utilisé pour effectuer le chiffrement et le déchiffrement. En d'autres termes, la clé de chiffrement est la "clé" réelle utilisée pour sécuriser les données, tandis que l'algorithme de chiffrement définit la façon dont cette clé est utilisée pour effectuer les opérations de chiffrement et de déchiffrement.

3. Différence entre une clé en tant que mot passe et une clé en tant que fichier dans le chiffrement AES avec openssl :

Dans le chiffrement AES avec OpenSSL, une clé peut être spécifiée soit en tant que mot de passe (par exemple via l'option `-pass`), soit en tant que fichier contenant la clé. Lorsqu'une clé est spécifiée en tant que mot de passe, OpenSSL utilise une dérivation de clé (comme PBKDF2) pour générer une clé à partir du mot de passe. Lorsqu'une clé est spécifiée en tant que fichier, OpenSSL lit simplement la clé à partir du fichier.

4. Créons un fichier contenant la clé secrète avec la commande : `openssl rand -out secret.key 32`

5. Chiffrons/déchiffrons une photo avec cette clé en utilisant l'algorithme AES-256-CBC :

Chiffrons la photo avec la commande : `openssl enc -aes-256-cbc -salt -in download.jpg -out photo_chiffre.enc -pass file:secret.key`

Déchiffrons la photo avec la commande : `openssl enc -d -aes-256-cbc -in photo_chiffre.enc -out photo_dechiffre.jpg -pass file:secret.key`

6. Comment partager une clé secrète :

Il existe plusieurs méthodes pour partager une clé secrète de manière sécurisée :

Canal sécurisé : On utilise un canal sécurisé tel que HTTPS, SFTP ou SCP pour transmettre la clé secrète. On s'assure que le canal de communication est chiffré et authentifié pour empêcher les interceptions et les attaques de l'homme du milieu.

Chiffrement asymétrique : Utilisez la cryptographie asymétrique (par exemple, RSA) pour chiffrer la clé secrète avant de l'envoyer. La personne qui souhaite partager la clé peut chiffrer la clé secrète avec la clé publique du destinataire. Une fois chiffrée, la clé secrète peut être envoyée sur un canal non sécurisé sans compromettre sa sécurité. Le destinataire peut ensuite utiliser sa clé privée pour déchiffrer la clé secrète.

Échange de clés Diffie-Hellman : Les deux parties peuvent utiliser le protocole de l'échange de clés Diffie-Hellman pour convenir d'une clé secrète commune sans jamais avoir besoin de transmettre cette clé sur le réseau. Le protocole permet à chaque partie de générer une clé publique et d'échanger ces clés publiques de manière sécurisée. À partir de ces clés publiques, une clé secrète commune peut être calculée de manière sécurisée.

Tp4 : Fonctionnement de TLS

Partie 1 : Fonctionnement TLS

1. Expliquons le fonctionnement de TLS avec un schéma :

Client	Serveur
Hello	
	Hello
Certificate	
	Certificate
Key Exchange	
	Key exchange
Finished	
	Finished
Secure Data Exchange	

1. Hello (Bonjour) : Le client envoie un message "Hello" au serveur pour initier la connexion TLS.

2. Certificate (Certificat) : Le serveur envoie son certificat au client, prouvant son identité.

3. Key Exchange (Échange de clés) : Le client et le serveur échangent des clés secrètes qui seront utilisées pour chiffrer et déchiffrer les données.

4. Finished (Terminé) : Les deux parties s'échangent des messages "Finished" pour confirmer que la connexion est établie de manière sécurisée.

5. Secure Data Exchange (Échange de données sécurisé) : Une fois la connexion établie, les données peuvent être échangées de manière sécurisée entre le client et le serveur

2. Dans le contexte de TLS comment le client vérifie l'authenticité du serveur ?

Pour vérifier l'authenticité du serveur dans le cadre de TLS, le client vérifie les éléments suivants :

L'identité du serveur, généralement vérifiée en comparant le nom de domaine du serveur avec celui indiqué dans le certificat du serveur.

L'authenticité du certificat du serveur, en vérifiant sa signature numérique à l'aide de l'autorité de certification (CA) de confiance du client.

La validité du certificat, en s'assurant qu'il n'a pas expiré et qu'il est émis pour le bon domaine.

3. Définition d'une suite cryptographique :

Une suite cryptographique est un ensemble d'algorithmes cryptographiques utilisés ensemble pour fournir des services de sécurité dans un protocole de communication sécurisée, comme TLS. Ces algorithmes incluent généralement des méthodes d'échange de clés, d'authentification, de chiffrement et d'intégrité des données.

4. Quelles sont les différentes parties essentielles d'une suite cryptographique :

Les différentes parties essentielles d'une suite cryptographique sont : L'algorithme d'échange de clés pour établir une clé de session sécurisée entre le client et le serveur.

L'algorithme d'authentification pour vérifier l'identité des parties communiquant.

L'algorithme de chiffrement symétrique pour chiffrer les données échangées entre le client et le serveur.

L'algorithme d'authentification de message pour vérifier l'intégrité des données échangées.

5.a- Citons quelques exemples de suites cryptographiques et leurs composants:

Suite TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 :

Algorithme d'échange de clés : ECDHE (Ephemeral Elliptic Curve Diffie-Hellman)

Algorithme d'authentification : RSA

Algorithme de chiffrement symétrique : AES-256-GCM

Algorithme d'authentification de message : SHA-384

Suite TLS_DHE_RSA_WITH_AES_128_CBC_SHA256 :

Algorithme d'échange de clés : DHE (Diffie-Hellman Ephemeral)

Algorithme d'authentification : RSA

Algorithme de chiffrement symétrique : AES-128-CBC

Algorithme d'authentification de message : SHA-256

5.b- Allez sur le site <https://ciphersuite.info/> pour voir les suites faibles, sûres, et recommandées :

Partie 2 : Applications (mkcert+Apache2)

1. Installons les prérequis :

2. Téléchargeons l'outil mkcert :

`wget https://github.com/FiloSottile/mkcert/releases/download/v1.4.3/mkcert-v1.4.3-linux-amd64`

3. Déplaçons et renommons le fichier :

4. Affichons le dossier contenant le certificat et la clé privée de l'autorité avec un utilisateur simple :

5. Avec un utilisateur simple créons un certificat et une clé privée pour le serveur apache pour les domaines localhost et 127.0.0.1 :

Dans votre cas, vous remplacerez 127.0.0.1 par l'adresse IP de votre machine (c'est obligatoire).

6. Avec un utilisateur simple installons le certificat de l'autorité local (rootCA.pem) dans le magasin des autorités de certification racines de confiance du système :

7. Nous remarquons que la clé privée et le certificat du serveur (qui vont être spécifiés dans le site virtuel) sont différents de ceux de l'autorité :

8. Créons un site virtuel ssl en copiant le fichier de configuration par défaut :

9. Renseignons les paramètres minimaux pour l'activation de https pour le site en question :

10. Activons le site virtuel et le module ssl :

11. Créons le fichier index de notre site web :

12. Utilisons `https://localhost` pour Accéder à notre site :

En utilisant `HTTPS://localhost` pour accéder au site, nous ne verrons pas l'exception de sécurité car mkcert crée des certificats auto-signés mais les installe également dans le magasin de certificats de confiance de notre système d'exploitation. Donc il n'affiche pas d'erreur de sécurité.

13. Sur le cadenas il n'y a pas aussi le point d'exclamation :

14. Remplaçons localhost par 127.0.0.1 et faisons le constat :

Lorsqu'on remplace localhost par 127.0.0.1 dans l'URL, On remarque qu'il y'a des messages d'avertissement liés à la sécurité. Car le certificat généré par mkcert ne correspond plus au domaine dans l'URL, Le certificat n'est valide que pour localhost. Cela entraîne une erreur car le certificat ne correspond pas au domaine spécifié dans la barre d'adresse du navigateur.

15. Dans ce cas de test on voit l'erreur `ssl_error_bad_cert_domain`. Qu'est-ce cela veut dire :

L'erreur `ssl_error_bad_cert_domain` signifie que le nom de domaine dans le certificat SSL ne correspond pas au nom de domaine auquel on essaie d'accéder. En d'autres termes, le certificat SSL est émis pour un domaine spécifique qui est localhost, mais nous essayons d'accéder au site via un autre domaine qui est 127.0.0.1.

16. Pourquoi voit-on sur le cadenas le message : Votre connexion à ce site n'est pas sécurisée ?

Le navigateur affiche un avertissement indiquant que la connexion n'est pas sécurisée car le certificat SSL ne correspond pas au domaine spécifié dans l'URL.

17. Utilisons une machine Windows pour accéder à notre site :

18. Une exception de sécurité liée à l'absence de l'autorité de confiance. Ajoutons le certificat de l'autorité dans le magasin des autorités de certification racines de confiance :

Exportons le certificat :

Ajoutons le certificat dans le magasin des autorités de certification racines de confiance :

Notre certificat est ajouté dans le magasin de certification :

18. Redémarrons le navigateur et donnons nos remarques :

Une fois que vous avez importé avec succès le certificat de l'autorité locale dans le magasin des autorités de certification racines de confiance de Windows et redémarré votre navigateur, on doit pouvoir accéder à votre site sans rencontrer d'exception de sécurité liée à l'absence de l'autorité de confiance. Cela signifie que le certificat émis par notre autorité locale est maintenant considéré comme valide et fiable par notre système d'exploitation et notre navigateur, ce qui permet d'établir une connexion HTTPS sécurisée sans afficher d'avertissements ou d'erreurs de certificat.

19. Faire un mini programme en PHP avec une page de connexion qui après authentification autorise ou refuse l'accès :

Créons la base de données où l'on stockera les utilisateurs :

Créons la table et insérons des enregistrements :

Affichons les enregistrements de la table users :

Donnons tous les privilèges à la base de données à l'utilisateur falilou :

Ecrivons le code php pour la page d'authentification qui utilisera la base données auth pour vérifier les utilisateurs. Si l'utilisateur est dans la base de données le site affiche « vous pouvez accéder au site » sinon il affiche « Vous ne pouvez pas accéder au site » :

Accédons au site d'authentification avec localhost/auth.php :

Connectons-nous avec falilou qui se trouve dans la base de données :

Connectons-nous avec un utilisateur au hasard qui ne se trouve pas dans la base de données :

20. Sécurisons le site d'authentification :

Copions notre configuration précédente en auth.conf :

Ajoutons notre fichier index dans la configuration :

Activons le site :

Accédons au site avec https :

Connectons-nous avec mariam :

Vérifions avec Wireshark si les données sont cryptées :